

Neural Network - Implementation in Java

Daniel K. Ayivon - kdayivon

July 10, 2026

Abstract

This work serves as compedium of machine learning knowledge, in which fundamentals ideas of Linear Algebra, Calculus, and programming paradigms are explored for deeper comprehension.

1 Perceptron - Neural Model

In machine learning, a perceptron can be broadly defined as an "artificial neuron"—a variable that holds a value. A *Multi-Layer Perceptron*, or *Neural Network*, is a programming paradigm



Figure 1: Simple representation of a Perceptron

where each layer of the network is composed of multiple perceptrons, each holding a value, akin to vectors. The network is usually divided into three distinctive but connected parts :

- The input layer - can also be considered as the 0th activation layer. Each perceptron in this layer holds a input—generally a float, given by the training data.
- The hidden layer - composed of n layers, each perceptron holding the value of a function where the parameters is each individual perceptron, weights, and biases of the previous layer.
- The output layer - the expected outputs given by the instance of data.

We can conceptually assume that the hidden layers of a neural network split the inputs into different recognizable patterns, which could combine into the expected output.

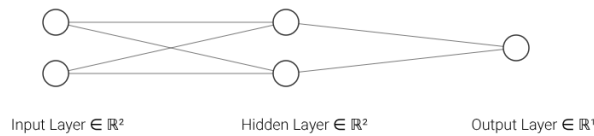


Figure 2: Simple representation of one neuron with one input

1.1 Forwarding

The notion of *forwarding* in an artificial neural network, is the process of sending information through the layers, from one layer to the next without forming loops or cycles. In a forward-feeding network, each perceptron in the next layer is a function of sum that adds up all the perceptron in the layer before it (1)

$$\sum_{i=1}^n w_i a_i \quad (1)$$

where n the number of perceptron in the previous layer, w the weights (or parameters) of the model, a the activations or inputs of the model. The function in its current state can return any value $k \in \mathbb{R}$, prompting the need to compress (or map) the values between an acceptable range, using functions known as *activation functions*. The most common activation functions are the **Sigmoid** and **ReLU** (Rectified Linear Unit) functions.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (\text{Sigmoid function})$$
$$ReLU(x) = \frac{x + |x|}{2} = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \quad (\text{ReLU function})$$

The final activation function (forwarding) is a measure of how positive the compressed value of this function is.

$$k = \sigma \left(\sum_{i=1}^n w_i a_i + b \right) \quad (2)$$

$$k = \sigma \left(\begin{bmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,n} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{k,1} & w_{k,2} & \cdots & w_{k,n} \end{bmatrix} \begin{bmatrix} a_1^0 \\ a_2^0 \\ \vdots \\ a_n^0 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_k \end{bmatrix} \right) \quad (3)$$

2 Machine Learning

In *Machine learning*, an elementary implementation of neural networks consists of finding an output y with a given input x , by adjusting different parameters in the equation

$$y = xw + b \quad (4)$$

where w is the weights, or connections between neurons, and b is the bias where each neuron has its own. For any random value given to the parameters, we must compute how different the output given by the equation and the expected output is.

2.1 Loss Function

In mathematical optimization and statistics, a *loss function* is a function that maps values of one or more parameters onto a real number that intuitively represents a "cost" associated with the event. It is used for parameter estimation, where the event is some function of the difference between the estimated and expected values for an instance of data. The finite goal of elementary machine learning is to drive the value of "cost" or "lost" function towards zero. Calculus helps us find the minimum of such a function using different linear methods, referred to as "*Gradient Descent*".

2.1.1 Naive Implementation - Twice

In the primitive machine learning example *Twice*, we computed the cost function as the sum of the square difference between the output of the model for each input and the expected output.

$$C(w) = \frac{1}{n} \sum_{i=1}^n (x_i w - y_i)^2 \quad (5)$$

If the value of the model (represented as $x_i w$) deviates too much from the expected value (y_i), squaring the difference magnifies the error. We divide by the number of samples n , to compute the average cost over the entirety of the sample

3 Numerical Gradients

The gradient of a function tells us the direction of steepest increase. In order to minimize the loss function, we compute the direction of the slope and move in the opposite. *Numerical gradients*, as opposed to *Analytical gradients*, are simple conceptually and in implementation, but pose quite a challenge as the parameter count increases. Numerical gradients approximate the derivative using discrete function values rather than analytical calculus, primarily via the **Finite Difference** methods, where choosing an erroneous step size h introduces numerical instability and precision issues. Numerical Gradients work by applying a small perturbation to the parameters and observing how much the function value changes.

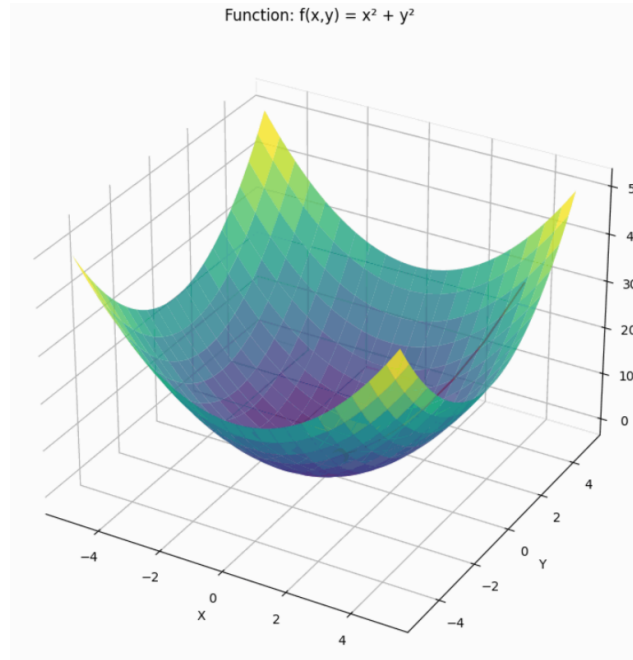


Figure 3: Gradient Descent - Numerical

3.1 Gradient Descent - Finite Difference

We use the finite difference to approximate the derivative of the function to hopefully find the global minimum; Since we randomize the starting point of the function (by randomizing the values of the weights and biases), there is a better chance to find a local minimum. As we arbitrarily choose a value that approaches 0 for the ϵ , to help find the direction of the tangent to the curve.

$$C'(w) = \frac{C(w + \epsilon) - C(w)}{\epsilon} \quad (6)$$

$$C'(w) = \lim_{\epsilon \rightarrow 0} \frac{C(w + \epsilon) - C(w)}{\epsilon} \quad (7)$$

3.2 Faster Cost Compute

Essentially, instead of computing the cost function, seeing the value and adjusting the parameters and seeing if the cost goes downward, we compute the velocity according to the value w , and keep subtracting from w the derivative. This is a faster and more accurate method as we don't need to specify a value that amounts to the step size down the function. We compute the derivative as the rule from calculus states

If a function is differentiable, its minima occurs at points where the derivative is zero.

$$C(w) = \frac{1}{n} \sum_{i=1}^n (x_i w - y_i)^2 \quad (8)$$

$$C'(w) = \left(\frac{1}{n} \sum_{i=1}^n (x_i w - y_i)^2 \right)' = \quad (9)$$

$$= \frac{1}{n} \left(\sum_{i=1}^n (x_i w - y_i)^2 \right)' \quad (10)$$

$$= \frac{2}{n} \sum_{i=1}^n (x_i w - y_i) x_i \quad (11)$$

$$(12)$$

As we're taking the derivative of a smooth quadratic function, it possesses a single global minimum where the derivative is null. The function also needs to be convex, meaning it needs to be twice-differentiable with its second derivative being nonnegative on its entire domain.

$$C''(w) = \frac{2}{n} \sum_{i=1}^n x_i^2 > 0 \quad (13)$$

The difference between this gradient descent algorithm and computing the finite difference, is that the model becomes too sensitive to the value ϵ we manually specified, making it hard to land on a specific value if the ϵ value is too big.

4 Analytical Gradient

4.1 neuron

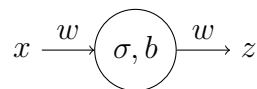


Figure 4: Simple representation of a Perceptron